

Distributed Observer, Trust Weighting, and Rollback

Quang Huy Nguyen

Slide 1: Observer Backbone

$$(\text{LO}_i) : \quad \hat{x}_0^{(i)}(t+1) = A_b \hat{x}_0^{(i)}(t) + B_b u_i(t) + F_i(y_i(t) - C_b \hat{x}_0^{(i)}(t))$$

$$(\text{DO}_i^{(j)}) : \quad \hat{x}_i^{(j)}(t+1) = A_b \left[\hat{x}_i^{(j)} + \sum_{l \in \mathcal{N}_i} w_{il}^{(j)} (\hat{x}_l^{(j)} - \hat{x}_i^{(j)}) + w_{i0}^{(j)} (\hat{x}_0^{(j)} - \hat{x}_i^{(j)}) \right] + B_b \hat{u}_j$$

The paper starts with two layers. The local observer estimates the directly measured vehicle. The distributed observer lets each host estimate target vehicle j by mixing its own estimate, neighbor estimates, and the direct local anchor $\hat{x}_0^{(j)}$. The key design variables are the weights $w_{il}^{(j)}(t)$ and $w_{i0}^{(j)}(t)$.

Slide 2: Error Dynamics and Boundedness

Define

$$e_i^{(j)} = \hat{x}_i^{(j)} - x^{(j)}, \quad e_0^{(j)} = \hat{x}_0^{(j)} - x^{(j)}, \quad \Delta u_j = u_j - \hat{u}_j.$$

If neighbor l sends corrupted data,

$$\tilde{x}_l^{(j)} = \hat{x}_l^{(j)} + a_l^{(j)}.$$

Then

$$e_i^{(j)}(t+1) = A_b \left[\bar{w}_{ii}^{(j)} e_i^{(j)} + \sum_{l \in \mathcal{N}_i} w_{il}^{(j)} e_l^{(j)} + w_{i0}^{(j)} e_0^{(j)} + \sum_{l \in \mathcal{A}_i} w_{il}^{(j)} a_l^{(j)} \right] - B_b \Delta u_j + d_j,$$

where

$$\bar{w}_{ii}^{(j)} = 1 - w_{i0}^{(j)} - \sum_{l \in \mathcal{N}_i} w_{il}^{(j)}.$$

$$\|e(t+1)\| \leq \rho \|e(t)\| + c_0 \|e_0(t)\| + c_u \|\Delta u(t)\| + c_d \|d(t)\| + c_a \sum_{l \in \mathcal{A}} w_{il}^{(j)}(t) \|a_l(t)\|.$$

Slide 3: Bounded Error Conclusion

If $\rho < 1$ and the local observer error, model mismatch, and attacks are bounded,

$$\limsup_{t \rightarrow \infty} \|e(t)\| \leq \frac{c_0 \bar{e}_0 + c_u \bar{\Delta} u + c_d \bar{d} + c_a \bar{a} \limsup_t \sum_{l \in \mathcal{A}} w_{il}^{(j)}(t)}{1 - \rho}.$$

This is the reason the estimation error remains bounded. The nominal consensus term is contractive. The local observer error and input mismatch are bounded. Malicious data only enters after multiplication by the trust weights. When trust drops, the malicious weight becomes small, so the attack influence is attenuated.

Slide 4: Paper Weight Rule

Trusted candidate set:

$$\mathcal{L}_i^{(j)}(t) = \{l \in \mathcal{N}_i : O_i(l, t) \geq \theta_{\min}\} \cup \{0 : LT_i^{(j)}(t) \geq \theta_{\min}\}.$$

Paper-like normalization:

$$n_i^{(j)}(t) = \max\{\kappa, |\mathcal{L}_i^{(j)}(t)| + 1\}.$$

Weights:

$$w_{il, \text{paper}}^{(j)}(t) = \begin{cases} 1/n_i^{(j)}(t), & l \in \mathcal{L}_i^{(j)}(t), \\ 0, & \text{otherwise,} \end{cases}$$

Self residual:

$$w_{ii, \text{paper}}^{(j)}(t) = 1 - w_{i0, \text{paper}}^{(j)}(t) - \sum_{l \in \mathcal{N}_i} w_{il, \text{paper}}^{(j)}(t).$$

Slide 5: Trust-Proportional Code Extension

The code extends the paper rule by using trust-proportional weights instead of equal weights.

$$\mathcal{C}_i^{(j)}(t) = \text{Top}_\kappa\{l : j \in \text{estimate}_l, l \neq i, l \neq j, T_i(l, t) \geq \theta_{\min}\}.$$

$$\alpha_0 = w0_{\text{fixed}}, \quad \alpha_s = w_{\text{self,base}}, \quad \alpha_N = \max(0, 1 - \alpha_0 - \alpha_s).$$

$$b_l(t) = \frac{T_i(l, t)}{\sum_{m \in \mathcal{C}_i^{(j)}} T_i(m, t)}.$$

$$r_0 = \alpha_0 L T_i^{(j)}(t) \phi_0, \quad r_s = \alpha_s \phi_s, \quad r_l = \alpha_N \phi_N b_l.$$

$$Z = r_0 + r_s + \sum_{l \in \mathcal{C}_i^{(j)}} r_l,$$

$$w_{i0}^{(j)} = \frac{r_0}{Z}, \quad w_{ii}^{(j)} = \frac{r_s}{Z}, \quad w_{il}^{(j)} = \frac{r_l}{Z}.$$

Slide 6: Mixed Paper-Code Weight and Rollback

Use a conservative blend:

$$w_{\text{mix}}^{(j)}(t) = (1 - \lambda) w_{\text{paper}}^{(j)}(t) + \lambda w_{\text{code}}^{(j)}(t), \quad 0 \leq \lambda \leq 1.$$

Then smooth over time:

$$w^{(j)}(t) = (1 - \eta) w^{(j)}(t - 1) + \eta w_{\text{mix}}^{(j)}(t).$$

With small η , for example:

$$\eta = 0.15.$$

Optional per-neighbor cap:

$$w_{il}^{(j)}(t) \leq w_{\text{max}}.$$

Overflow goes to remaining trusted neighbors or to self:

$$w_{ii}^{(j)}(t) \leftarrow 1 - w_{i0}^{(j)}(t) - \sum_l w_{il}^{(j)}(t).$$

Let $\mathcal{M}(t)$ be detected malicious IDs. Starting from previous clean state,

$$x_{\text{prev}} = \hat{x}_i^{(j)}(t).$$

Clean correction:

$$\Delta x_{\text{clean}} = \mathbf{1}_{\{0 \notin \mathcal{M}\}} w_{i0}^{(j)}(z_0 - x_{\text{prev}}) + \sum_{l \notin \mathcal{M}} w_{il}^{(j)}(z_l - x_{\text{prev}}).$$

Constrained consensus replay:

$$x_{\text{cons}} = \Pi_{\mathcal{X}}(x_{\text{prev}} + \Delta x_{\text{clean}}).$$

Prediction replay:

$$\hat{x}_{\text{clean}}^{(j)}(t + 1) = \Pi_{\mathcal{X}}(f_{\text{pred}}(x_{\text{cons}}, u, dt)).$$

Rollback Explanation

For rollback, explain it as a **repair step after the trust detector has already found bad contributors**.

The normal observer update mixes several sources:

$$\hat{x}_i^{(j)}(t+1) = \text{prediction after} \left[\hat{x}_i^{(j)}(t) + w_{i0}^{(j)}(\hat{x}_0^{(j)} - \hat{x}_i^{(j)}) + \sum_{l \in \mathcal{N}_i} w_{il}^{(j)}(\hat{x}_l^{(j)} - \hat{x}_i^{(j)}) \right].$$

If one neighbor is malicious, its corrupted estimate enters through:

$$w_{il}^{(j)}(\tilde{x}_l^{(j)} - \hat{x}_i^{(j)}).$$

The trust weights reduce this influence, but rollback goes further: it **removes the malicious contribution completely and rebuilds the update**.

In the code, this is the method:

`contamination_rollback.py`

line 376: `_replay_without_malicious(...)`.

The idea is:

$$\mathcal{M}(t) = \text{set of detected malicious contributors.}$$

Start from the previous clean state:

$$x_{\text{prev}} = \hat{x}_i^{(j)}(t).$$

Then rebuild the correction using only clean sources:

$$\Delta x_{\text{clean}} = \mathbf{1}_{0 \notin \mathcal{M}} w_{i0}^{(j)}(\hat{x}_0^{(j)} - x_{\text{prev}}) + \sum_{l \in \mathcal{N}_i, l \notin \mathcal{M}} w_{il}^{(j)}(\hat{x}_l^{(j)} - x_{\text{prev}}).$$

Then compute the clean consensus state:

$$x_{\text{cons}} = x_{\text{prev}} + \Delta x_{\text{clean}}.$$

The code also applies physical constraints:

$$x_{\text{cons}} \leftarrow \Pi_{\mathcal{X}}(x_{\text{cons}}).$$

Here, $\Pi_{\mathcal{X}}$ means "project back to physically valid values," for example heading wrapping, velocity bounds, and acceleration bounds.

Finally, if a prediction model is available, the code propagates the repaired consensus state:

$$\hat{x}_{\text{clean}}^{(j)}(t+1) = \Pi_{\mathcal{X}}(f_{\text{pred}}(x_{\text{cons}}, u, dt)).$$

Simple talk notes:

"The trust-weighted observer already reduces the effect of malicious data by giving low-trust agents small weights. But rollback is stronger. After an agent is detected as malicious, we do not only reduce its weight; we remove its past contribution from the current update. The algorithm goes back to the previous trusted state, recomputes the consensus correction using only non-malicious direct and neighbor sources, applies physical constraints, and then reruns the prediction step. So the observer is not only attack-aware during fusion, it can also repair a contaminated estimate after detection."

Short version for slide:

$$\hat{x}_{\text{rollback}}^{(j)}(t+1) = \Pi_{\mathcal{X}} \left[f_{\text{pred}} \left(\Pi_{\mathcal{X}} \left(\hat{x}_i^{(j)}(t) + \sum_{s \notin \mathcal{M}} w_s^{(j)}(z_s^{(j)} - \hat{x}_i^{(j)}(t)) \right), u, dt \right) \right]$$

where s represents the direct source 0 and trusted neighbors l .